

IFQ557: Assignment 2 Declaration

Section 1: Team declaration

Please declare your team details on this submission itself by completing and including the table below. The details should reflect the same team as defined in your canvas group. It will assist the marker.

Functional team declaration

Functional Team		
Group/team number:	Group 01	
GitHub name and link:	Leamon-Lee/IFQ557-Assignment-2	
Full name	Student#	Email
Jingru (Leamon) Li	2422441049	2422441049@stu.jit.edu.cn
Yicheng (Bacon) Rong	2422441046	2422441046@stu.jit.edu.cn
Taohao (Leaf) Ye	2422441048	yetaohao4@gmail.com
Zhengyan (Paul) Zhang	2422441050	zzy20051022zzy123@gmail.com

Section 2: Team contribution statement

Brief overview from each team member explaining how they contributed: State your role within the team and the tasks that you completed. There is no word limit, however, 60-100 words should be sufficient.

Jingru (Leamon) Li	Created the initial Flask MVC scaffold, merged the frontend prototype into Flask views/templates, wrote README and submission documentation, developed the value object layer and validation rules, refactored models to use value objects, connected frontend API routes, and stabilized database rebuild/startup behaviour.
Yicheng (Bacon) Rong	Implemented and polished the frontend experience for SoundWave Events, including the homepage prototype, loading entrance page, hero carousel, footer, visual styling, role-aware UI behaviour, organizer dashboard, event display fixes, and ticket page presentation. Also connected frontend expectations to API contracts and helped prepare the interface for final submission.
Taohao (Leaf) Ye	Completed major backend and integration work for authentication, registration, event management, comments, ticket booking, user profile APIs, organizer event status changes, seed data, image handling, password hashing migration, and form submission fixes. Also added and refined tests and corrected several route and pricing bugs during integration.
Zhengyan (Paul) Zhang	Built the early database and model foundation, including core user, participant, and organizer model definitions supporting login and registration tests. This work helped establish the SQLAlchemy data layer that later features, services, and tests extended.

Section 3: git shortlog > output.txt

Leamon Lee (15):

- Initial Flask MVC scaffold
- Merge bacon frontend into Flask views
- Add project README and UML guide
- Rename site to SoundWave Events
- Refactor Paul models with value objects
- Apply value objects across domain models
- Require value objects in user roles
- Complete value object constraints from UML
- Add detailed submission notes
- Merge leamon value objects into temp
- Refactor models to use value objects
- Add database rebuild toggle to runner
- Connect frontend API routes
- Stabilize database rebuild runner
- Add latex assignment files

rongyicheng23-afk (16):

- Add SoundWave event prototype
- Refine homepage hero carousel
- Add polished site footer
- Prepare frontend for final submission
- Merge remote-tracking branch 'origin/main' into bacon
- Add SoundWave loading entrance page
- Refine loading page with dynamic concert hero
- Polish loading page typography
- Increase loading page background impact
- Use custom generated loading background
- Replace loading video with smooth realtime ambience
- Refine loading page ambience without line effects
- Add dramatic animated concert light show
- Optimize loading light show performance
- Refine hero actions and auto-dismiss alerts
- Add role-based auth UI and API contract

yetaohao (17):

- 登录, 注册, 登出功能完善, 修改了 login,signup 部分前端代码
(English: Improved login, registration, and logout; updated login/signup frontend code)
- 前端页面修改: 原数据写死改为从数据库中获取动态数据, 新增错误处理页面
(English: Changed frontend pages from hard-coded data to dynamic database data; added error pages)
- 根据最新 models 完善用户认证和事件管理功能, 重构相关模型和路由, 更新模板以提升用户体验
(English: Improved authentication and event management based on latest models; refactored
↳ models/routes/templates)
- 对上一步提交的一些补充
(English: Added follow-up changes for the previous commit)
- Revert " 对上一步提交的一些补充"
(English: Reverted the follow-up changes from the previous commit)
- Revert " 根据最新 models 完善用户认证和事件管理功能, 重构相关模型和路由, 更新模板以提升用户体验"
(English: Reverted the authentication/event-management refactor based on latest models)
- 根据 Paul 最新版 models 完善用户模型和参与者模型, 添加事件创建、更新、取消及注册功能; 更新认证路由以支持用户登录和注册
(English: Updated user/participant models from Paul's latest models; added event
↳ create/update/cancel/registration and auth route support)
- 修改了一个冲突内容
(English: Fixed a merge conflict item)
- 错误拉取主分支后出现报错, 修正提交
(English: Fixed errors caused by accidentally pulling the main branch)
- feat: 新增评论系统和购票预订流程
(English: Added comment system and ticket-booking flow)
- feat: 新增创建/编辑活动功能
(English: Added create/edit event functionality)
- feat: 填充全部空壳页面
(English: Filled all placeholder pages)
- feat: 实现首页搜索/筛选和 Organizer 公告功能
(English: Implemented homepage search/filtering and Organizer announcements)
- feat: 新增 Organizer 取消活动功能
(English: Added Organizer event cancellation)
- 修正了文本模板切片操作失败的问题
(English: Fixed failed text-template slicing operation)
- 修复图片缺失问题
(English: Fixed missing image issue)

通过 pytest 进行的项目测试, 测试结果报告位于 test 文件夹中
(English: Ran project tests with pytest; test report is in the test folder)

```
yetaohao4-cmd (1):  
Merge branch 'main' into Leaf
```

张峥岩 (1):
feat: 完成 User, Participant, Organizer 模型定义, 支持登录注册测试
(English: Completed User, Participant, and Organizer model definitions supporting login/registration
↪ tests)

Section 4: Team use of AI statement

As a team, we report that some members used AI in an authorized way, as development and writing support for this assignment. AI output was not accepted automatically. Team members reviewed, edited, tested, and took responsibility for the final code, documentation, declaration content, and submission files.

Summary of AI use

AI tools were used to support brainstorming, debugging, refactoring, documentation drafting, LaTeX formatting, and interpretation of test output. They were also used to help summarize git history and repository evidence into clearer contribution statements. AI assistance was treated as advisory: the team checked suggestions against the actual Flask application, repository files, git history, and generated PDF before including the final material.

Key examples of AI use

- Prompt: "Based on this Flask music event project, help summarize each team member's contribution from the git history and repository files."
- Input/upload: Repository files including README, SUBMIT notes, tests, routes, models, templates, and git shortlog output.
- Output used: Draft contribution summaries for Jingru (Leamon) Li, Yicheng (Bacon) Rong, Taohao (Leaf) Ye, and Zhengyan (Paul) Zhang. The team checked the summaries against the repository and edited them before placing them in Section 2.

===== NEXT SAMPLE OF AI USE =====

- Prompt: "Review this Flask route/model change and suggest possible bugs or missing validation before we commit it."
- Input/upload: Relevant snippets from Flask routes, SQLAlchemy models, forms, and error output from local testing.
- Output used: Suggestions about edge cases, validation checks, and likely causes of errors. The team manually inspected the code, selected relevant fixes, and verified behaviour with local testing before accepting any changes.

===== NEXT SAMPLE OF AI USE =====

- Prompt: "Help format this assignment declaration in LaTeX so that the tables are readable, aligned, and suitable for PDF submission."

- Input/upload: The declaration template, team details, contribution text, git history, and PDF layout notes.
- Output used: LaTeX formatting adjustments for tables, page breaks, links, and explanatory notes. The generated PDF was compiled and visually checked before inclusion.

===== NEXT SAMPLE OF AI USE =====

- Prompt: "Explain this pytest or runtime error and suggest likely files to inspect in the Flask project."
- Input/upload: Terminal error messages, selected test output, and related project files.
- Output used: Debugging guidance and possible causes. The team used the suggestions to guide manual investigation, then implemented and tested fixes themselves.